

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»



Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

Лабораторна робота № 10

з дисципліни «Алгоритмізація та програмування»

на тему: «Створення і обробка структур даних на мові C ++»

XAI.301.G12.311.8 ПР

Виконав: здобувач 1 курсу, групи 311
напряму підготовки (спеціальності) G12
Авіаційна та ракетно-космічна техніка
Максим ПАСНІЧЕНКО

Прийняла: доц. каф.301, канд. техн. наук,

_____ Олена ГАВРИЛЕНКО

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення структур (записів) на мові C ++, а також їх передачі в функції, і реалізувати оголошення і обробку структур на мові C ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити задачу зі структурами даних (задача **Param79 / Struct79**):

Описати тип TTime-запис з полями цілого типу Hour (години), Min (хвилини) і Sec (секунди). Описати функцію AddSec(T, N) типу TTime з двома вхідними параметрами типу TTime і цілого числа, яка змінює час T на +N секунд (якщо час T є неправильним, то він повертається без змін). За допомогою функції AddSec вивести новий час для п'яти заданих моментів часу.

Завдання 2. Вирішити задачі **Begin15** та **Boolean32**. Для кожної з цих задач необхідно:

- A.** Описати структуру, яка містить всі вхідні і всі вихідні дані задачі.
- B.** Визначити функцію (метод), що реалізує обробку структури відповідно до задачі з передачею параметрів за вказівником.
- C.** Викликати функцію (метод) після оголошення змінної і введення значень полів вхідних даних.
- D.** Вивести значення полів вихідних даних.

Умова задачі **Begin15**: Дана площа S кола. Знайти його діаметр D і довжину кола L , що обмежує це коло, з огляду на те, що $L = \pi \cdot D$, $S = \pi \cdot D^2/4$. Як значення π вважати рівним 3.14.

Умова задачі **Boolean32**: Дано цілі числа a , b , c , що є сторонами деякого трикутника. Перевірити істинність висловлювання: «Трикутник зі сторонами a , b , c є прямокутним».

Завдання 3. Завдання 1 і 2 реалізувати окремими функціями без параметрів. У головній функції main() організувати циклічне меню для багаторазового виконання завдань. Структурувати проєкт програми: винести оголошення структур та прототипи функцій у заголовний файл .h, а реалізацію логіки функцій-в окремий файл .cpp.

Завдання 4. Відобразити у звіті три діаграми активності: одну для функції обробки із Завдання 1 (AddSec), та дві для функцій Завдання 2 в цілому. Використовуючи засоби генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою заданих промптів.

1) «Ти-викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання

<середнього> рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

2) «Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у

5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

3) «Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань» Проаналізуйте задані питання, коментарі і оцінки, надані ШІ. Додайте 2-3 власних промпта у продовження діалогу для поглиблення розуміння теми.

ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Struct79. (Робота з часом)

Вхідні дані (ім'я, опис, тип, обмеження):

ім'я - T, опис - початковий час, тип - TTime, обмеження - години від 0 до 23, хвилини та секунди від 0 до 59.

ім'я-N, опис - секунди для додавання, тип - int, обмеження - відсутні.

Вихідні дані (ім'я, опис, тип):

ім'я - res, опис - оновлений або незмінний час, тип - TTime.

Алгоритм вирішення показано нижче/на рис. 1.1

Діаграма активності: Функція AddSec (Param79)

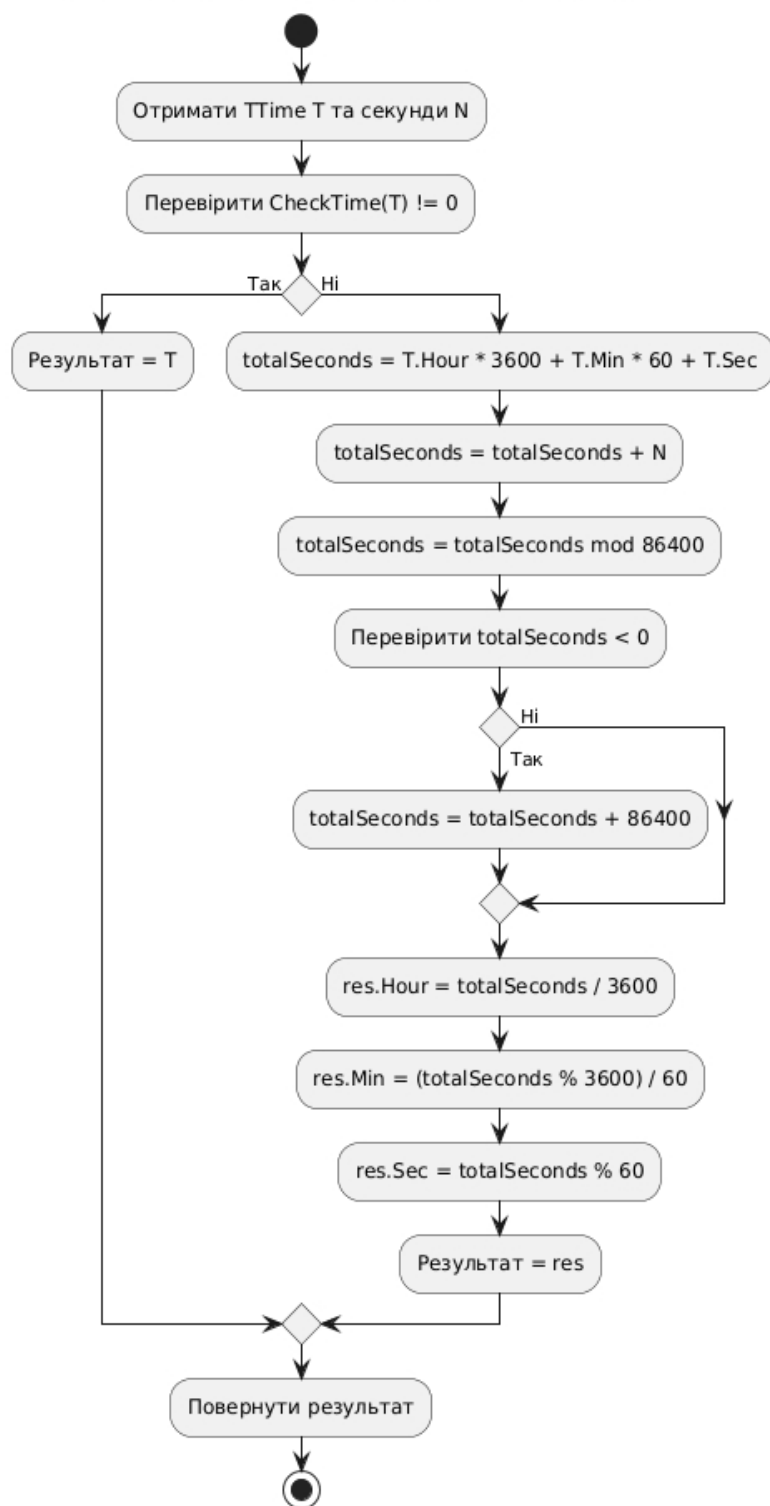


Рисунок 1.1 – Діаграма активності до задачі Param79

Лістинг коду вирішення задачі розділ і номер задачі наведено в дод. А (стор. 8).

Екран роботи програми показаний на рис. Б.1.1

Завдання 2.

Вирішення задачі Begin15 (Геометрія кола)

Вхідні дані (ім'я, опис, тип, обмеження):

ім'я - S, опис - площа кола, тип - double, обмеження - $S \geq 0$.

Вихідні дані (ім'я, опис, тип):

ім'я - D, опис - діаметр кола, тип - double.

ім'я-L, опис - довжина кола, тип - double.

Алгоритм вирішення показано нижче/на рис. 1.2

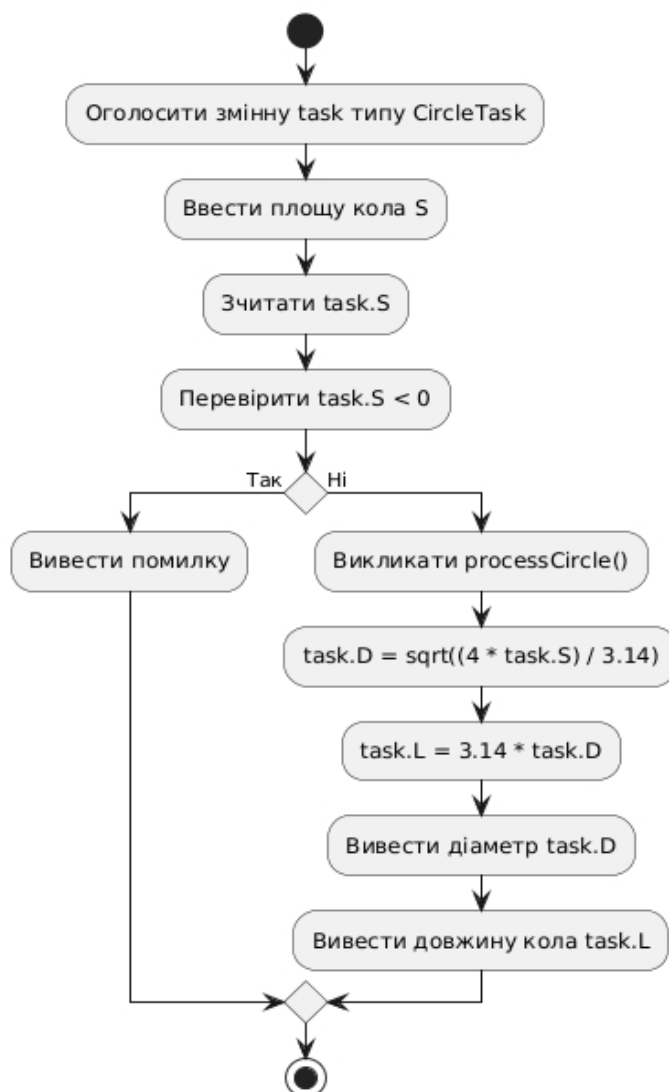
Діаграма активності: Завдання Begin15 (в цілому)

Рисунок 1.2 – Діаграма активності до задачі Begin15.

Лістинг коду вирішення задачі розділ і номер задачі наведено в дод. А (стор. 9).

Екран роботи програми показаний на рис. Б.1.2

Завдання 3.

Вирішення задачі Boolean32 (Перевірка трикутника)

Вхідні дані (ім'я, опис, тип, обмеження):

ім'я : a, b, c, опис - сторони трикутника, тип - int, обмеження - $a > 0$, $b > 0$, $c > 0$.

Вихідні дані (ім'я, опис, тип):

ім'я - isRight, опис - результат перевірки на прямокутність, тип - bool.

Алгоритм вирішення показано нижче/на рис. 1.3

Діаграма активності: Завдання Boolean32 (в цілому)

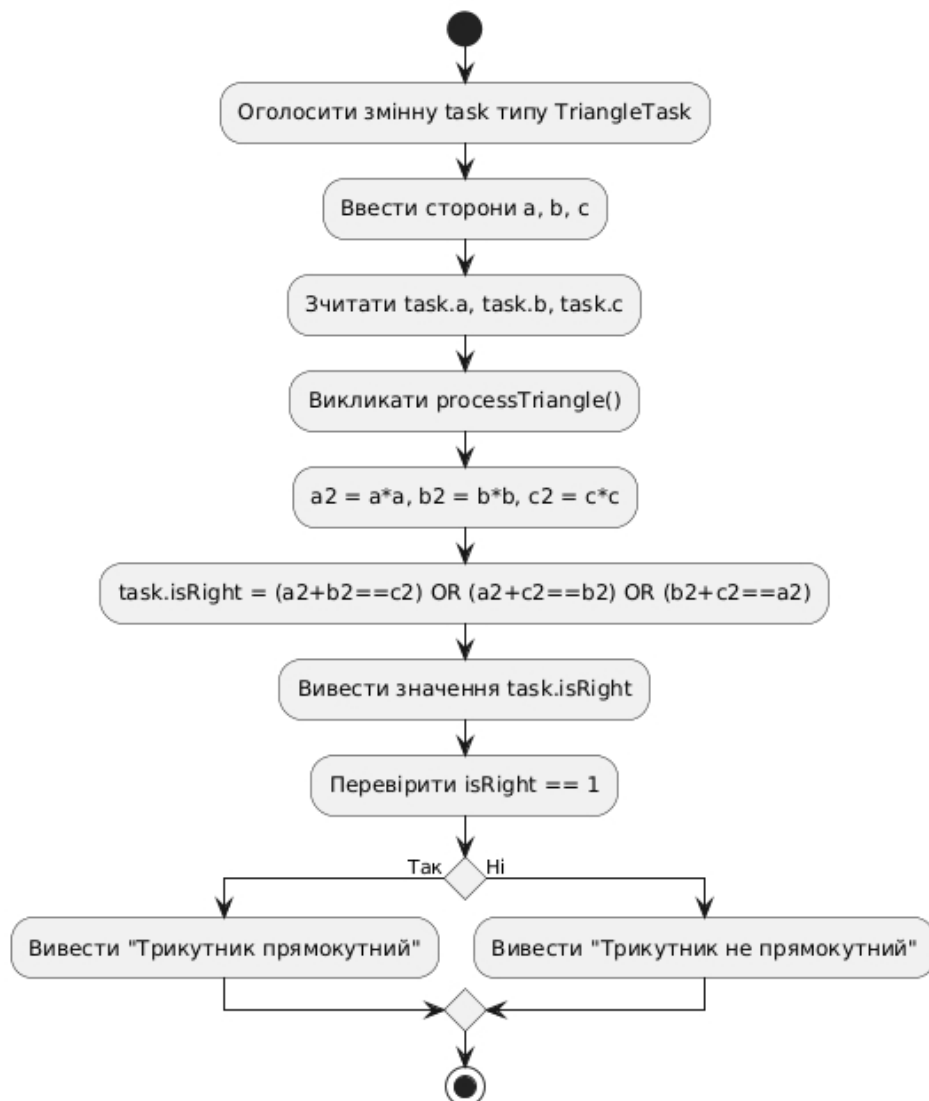


Рисунок 1.3 – Діаграма активності до задачі Boolean32.

Лістинг коду вирішення задачі розділ і номер задачі наведено в дод. А (стор. 9).

Екран роботи програми показаний на рис. Б.1.3

ВИСНОВКИ

Було вивчено теоретичні основи створення та використання структур даних мовою C++. Закріплено механізми передачі структур у функції як параметрів (зокрема за вказівником), а також відпрацьовано в коді програми принципи модульного програмування шляхом розділення проєкту на заголовні файли та файли реалізації. Крім того, отримано навички розробки інтерактивного консольного меню та побудови UML-діаграм і проведення самоаналізу.

ДОДАТОК А

Лістинг коду програми

фрагмент коду до Завдання 1 (Param79)

Код демонструє оголошення структури часу та алгоритм додавання секунд.

```

struct TTime {
    int Hour;
    int Min;
    int Sec;
};

// Перевірка правильності часу
int CheckTime(TTime T) {
    if (T.Hour < 0 || T.Hour > 23 || T.Min < 0 || T.Min > 59 || T.Sec < 0 || T.Sec
    > 59) {
        return 1; // Час неправильний
    }
    return 0; // Час правильний
}

// Основний алгоритм додавання секунд
TTime AddSec(TTime T, int N) {
    if (CheckTime(T) != 0) return T;

    long long totalSeconds = T.Hour * 3600 + T.Min * 60 + T.Sec;
    totalSeconds += N;

    totalSeconds = totalSeconds % 86400;
    if (totalSeconds < 0) totalSeconds += 86400;

    TTime res;
    res.Hour = totalSeconds / 3600;
    res.Min = (totalSeconds % 3600) / 60;
    res.Sec = totalSeconds % 60;

    return res;
}

```

фрагмент коду до Завдання 2.1 (Begin15)

фрагмент показує структуру з вхідними/вихідними даними та функцію обробки
кола через передачу параметрів за вказівником.


```

struct CircleTask {
    double S; // Вхідні дані: площа
    double D; // Вихідні дані: діаметр
    double L; // Вихідні дані: довжина кола
};

// Функція розрахунку, що приймає структуру за вказівником
void processCircle(CircleTask* c) {
    const double PI = 3.14;
    // Знаходимо діаметр з площі
    c->D = sqrt((4 * c->S) / PI);
    // Знаходимо довжину кола
    c->L = PI * c->D;
}

```

Фрагмент коду до Завдання 2.2 (Boolean32)

Тут наведено структуру для трикутника та алгоритм перевірки теореми
Піфагора .

```

struct TriangleTask {
    int a, b, c; // Вхідні дані: сторони
    bool isRight; // Вихідні дані: чи є прямокутним
};

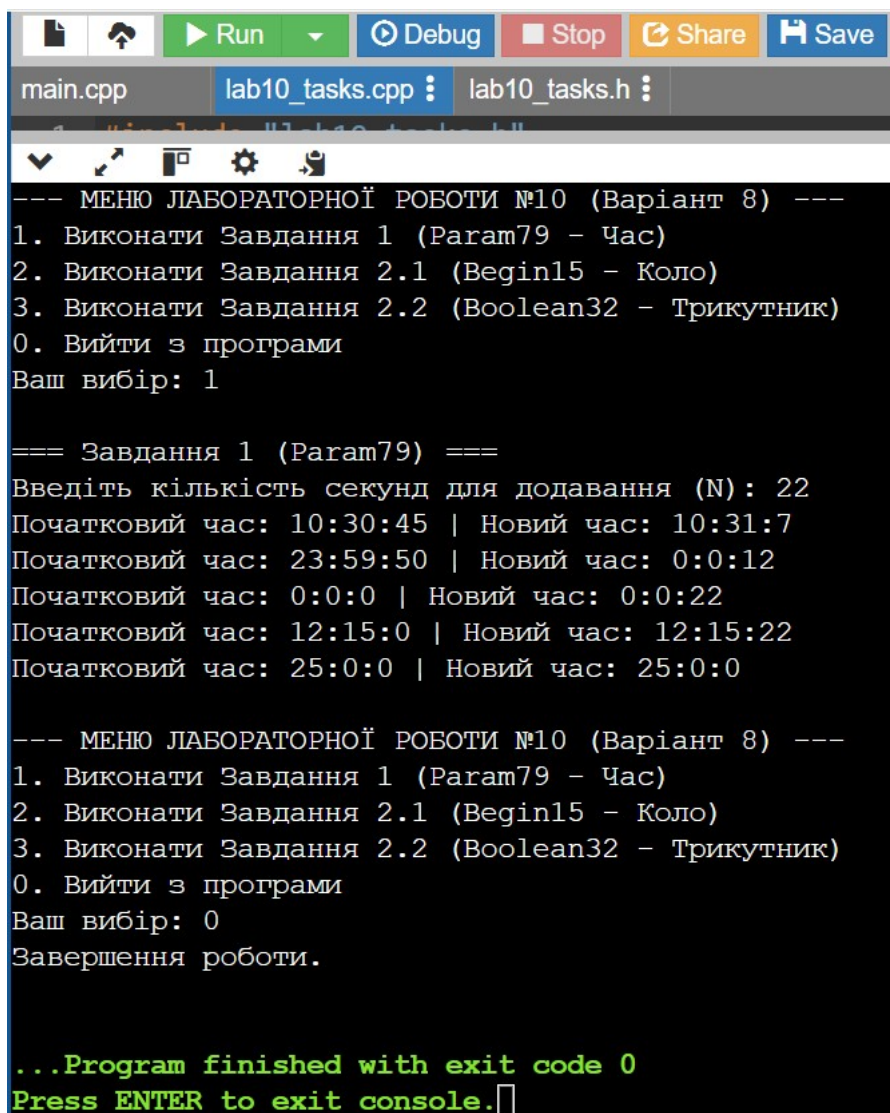
// Функція перевірки, що приймає структуру за вказівником
void processTriangle(TriangleTask* t) {
    // Зведення сторін у квадрат
    long long a2 = t->a * t->a;
    long long b2 = t->b * t->b;
    long long c2 = t->c * t->c;

    // Перевірка умови прямокутного трикутника
    t->isRight = (a2 + b2 == c2) || (a2 + c2 == b2) || (b2 + c2 == a2);
}

```

ДОДАТОК Б

Скрін-шоти вікна виконання програми



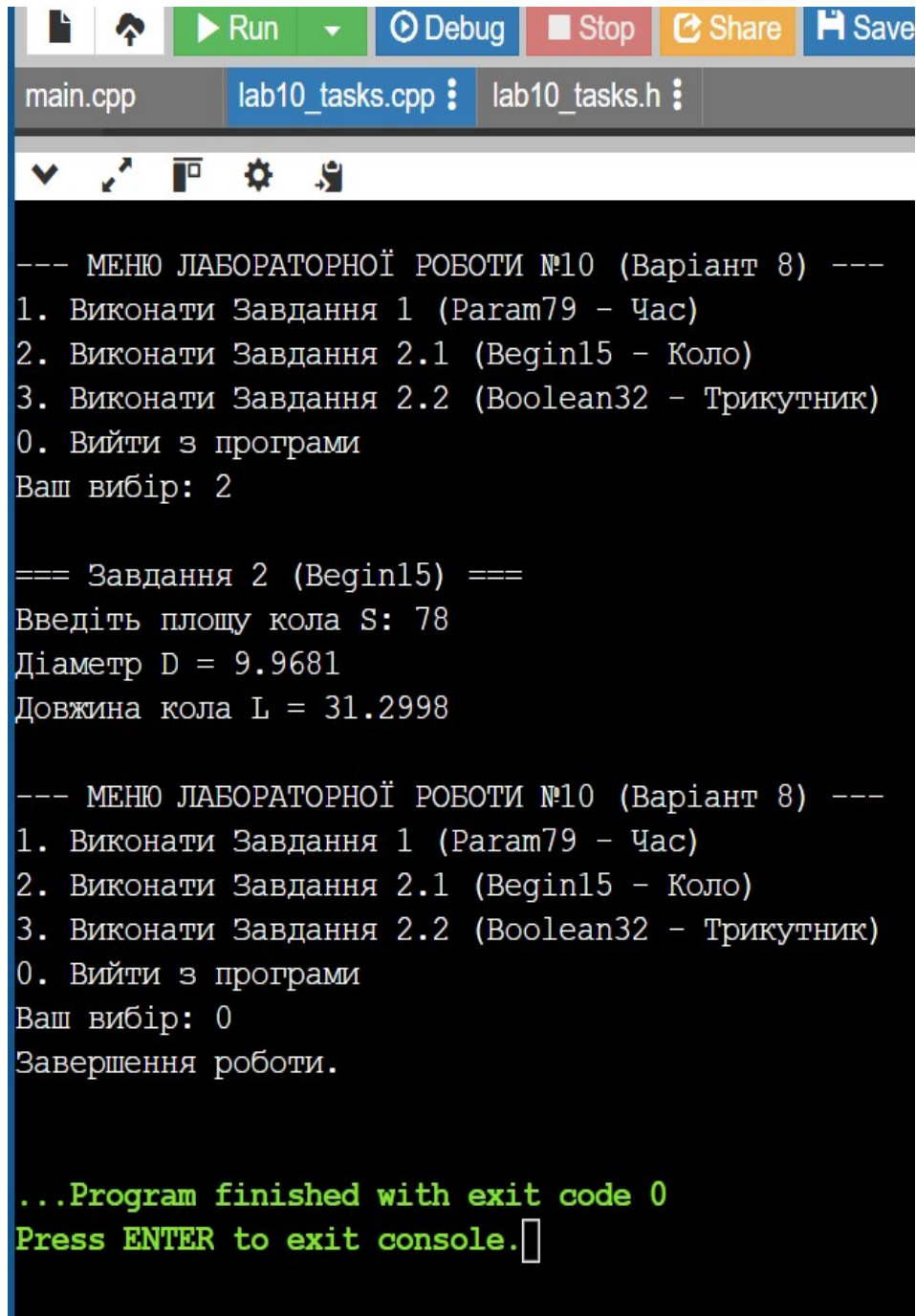
```
--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 1

=== Завдання 1 (Param79) ===
Введіть кількість секунд для додавання (N): 22
Початковий час: 10:30:45 | Новий час: 10:31:7
Початковий час: 23:59:50 | Новий час: 0:0:12
Початковий час: 0:0:0 | Новий час: 0:0:22
Початковий час: 12:15:0 | Новий час: 12:15:22
Початковий час: 25:0:0 | Новий час: 25:0:0

--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.1.1 – Екран виконання програми для вирішення завдання 1
Param79



```
main.cpp lab10_tasks.cpp lab10_tasks.h
--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 2

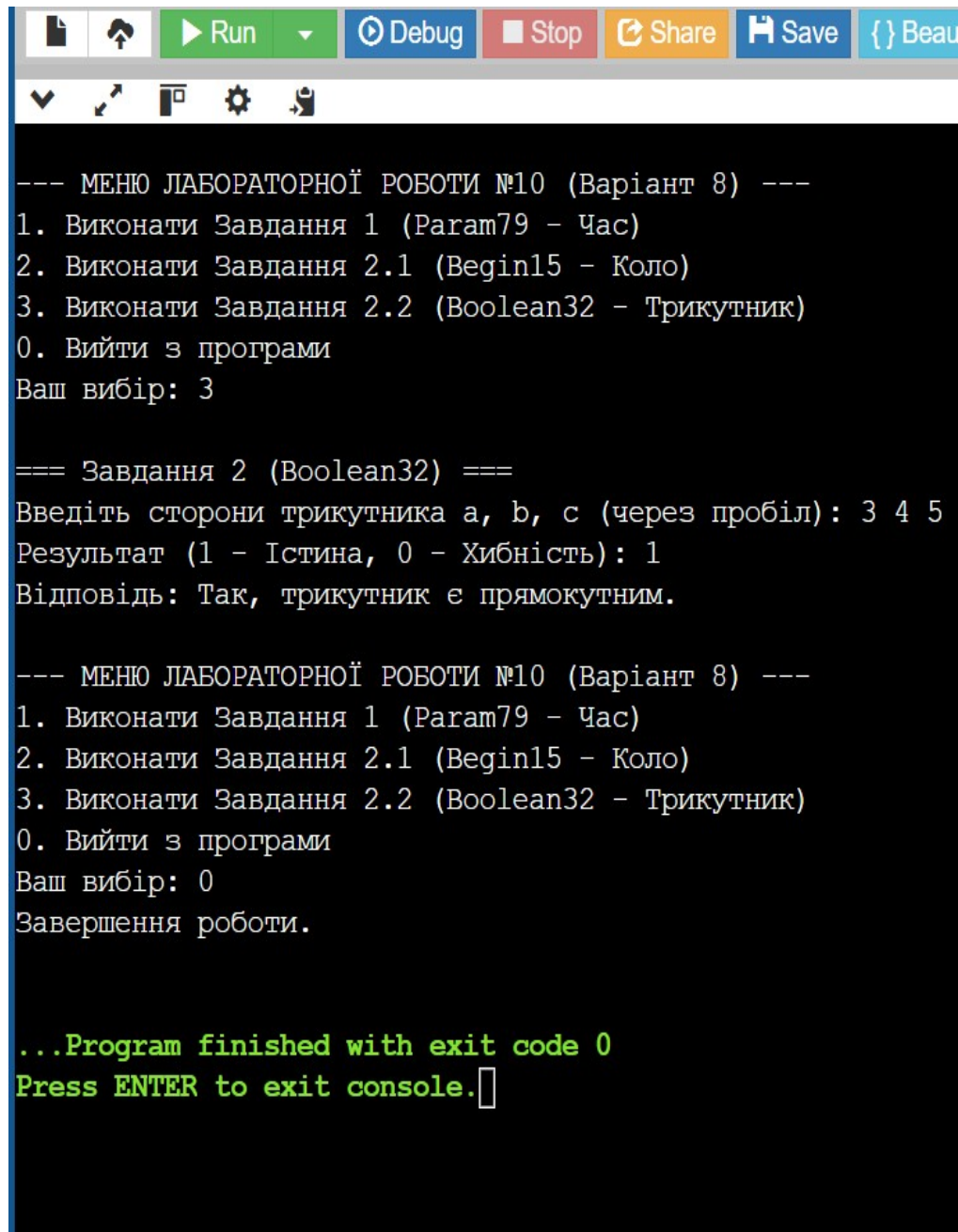
=== Завдання 2 (Begin15) ===
Введіть площу кола S: 78
Діаметр D = 9.9681
Довжина кола L = 31.2998

--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.1.2 – Екран виконання програми для вирішення завдання 2.1

Begin15



```
--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 3

=== Завдання 2 (Boolean32) ===
Введіть сторони трикутника a, b, c (через пробіл): 3 4 5
Результат (1 - Істина, 0 - Хибність): 1
Відповідь: Так, трикутник є прямокутним.

--- МЕНЮ ЛАБОРАТОРНОЇ РОБОТИ №10 (Варіант 8) ---
1. Виконати Завдання 1 (Param79 - Час)
2. Виконати Завдання 2.1 (Begin15 - Коло)
3. Виконати Завдання 2.2 (Boolean32 - Трикутник)
0. Вийти з програми
Ваш вибір: 0
Завершення роботи.

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок Б.1.3 – Екран виконання програми для вирішення завдання 2.2
Boolean32

ДОДАТОК В

Діалог з ШІ для самоаналізу

ЧАСТИНА 1: Завдання для самоаналізу

Користувач:

Використовуючи засоби генеративного ШІ, провести самоаналіз отриманих знань і навичок за допомогою заданих промптів...

ШІ:

(Генерація 5 тестових та 5 відкритих питань середнього рівня складності на основі наданого коду та теорії).

Тестові питання:

1. Яке призначення конструкції `#ifndef LAB10_TASKS_H ...` у заголовному файлі?

- А) Для підключення стандартних бібліотек.
- Б) Для запобігання повторному включенню файлу при компіляції.
- В) Для прискорення роботи програми.
- Г) Це вимога для роботи з вказівниками.

2. У функції `void processCircle(CircleTask c)` для доступу до поля `S` використовується:

- А) Оператор крапка: `c.S`
- Б) Оператор розіменування: `c.S`
- В) Оператор стрілка: `c->S`
- Г) Оператор адреси: `&c.S`

3. Ви передаєте структуру в `TTime AddSec(TTime T, int N)` за значенням. Що відбудеться з оригінальною змінною в `main`, якщо всередині функції змінити `T.Hour`?

- А) Зміниться і в `main`.
- Б) Залишиться незмінною, оскільки функція працює з копією.
- В) Буде помилка компіляції.
- Г) Значення обнулиться.

4. Яку інженерну проблему вирішує рядок `if (totalSeconds < 0) totalSeconds += 86400;?`

- А) Переведення годин у хвилини.
- Б) Запобігання виходу за межі типів даних.
- В) Коректний перехід через нуль (північ) при відніманні секунд.
- Г) Перевірку наявності високосного року.

5. Що передається у функцію під час виклику `processTriangle(&task)?`

- А) Копія всіх даних структури.
- Б) Вказівник на саму функцію.
- В) Адреса в оперативній пам'яті, де зберігається змінна `task`.
- Г) Логічне значення `true` або `false`.

Відкриті питання:

Проектування:

1. Чому в першому завданні ви передали структуру за значенням, а в другому — за вказівником?

2. Оптимізація типів: Чому для квадратів сторін використано тип `long long`, хоча самі сторони мають тип `int`?

3. Модульність: В який файл додається оголошення нової функції, а в який — її реалізація?

4. Масштабування: Як зміниться підхід, якщо треба обробити великий масив із 1000 трикутників?

5. Обробка помилок: Як покращити функцію `CheckTime`, щоб позбутися «магічних чисел» 0 та 1 для позначення помилок?

ЧАСТИНА 2: Відповіді на тести:

1-Б, 2-В, 3-Б, 4-В, 5-В.

Відповіді на відкриті питання:

1. Відповідь: Функція `AddSec` повинна порахувати і видати абсолютно новий час, не ламаючи старий. А функції `processCircle` та `processTriangle` мають записати пораховані результати (діаметр, довжину, прапорець) прямо в ту ж саму

структуру, яка вже створена. Вказівник допомагає працювати безпосередньо з оригіналом в пам'яті, щоб не створювати зайвих копій.

2. Відповідь: Якщо сторони трикутника будуть великими числами, то під час піднесення до квадрату їхнє значення перевищить ліміт звичайного типу `int`. Тоді відбудеться цілочисельне переповнення, число стане від'ємним, і вся теорема Піфагора порахується неправильно. `long long` дає великий запас пам'яті.

3. Відповідь: Оголошення структури та прототипу функції йде в заголовний файл `lab10_tasks.h`, а сам код з логікою — у файл реалізації `lab10_tasks.cpp`. Це потрібно, щоб компілятор бачив інтерфейс окремо від коду і не виникало помилок повторного визначення.

4. Відповідь: Замість передачі поодиноких адрес через `&task`, ми створимо масив або вектор структур. Тоді у функції ми будемо передавати цей масив через посилання, щоб перебирати елементи в циклі та не копіювати тисячу об'єктів у пам'яті заново.

5. Відповідь: Замість повернення `int` (де 1 — це помилка, а 0 — успіх) краще зробити тип повернення просто `bool`. Тоді функція буде називатися, наприклад, `IsTimeValid`. Вона повертатиме `true`, якщо час правильний, і `false`, якщо є помилка. Це прибере плутанину з інвертованою логікою.

Відповіді на додаткові промпти (Поглиблення теми):

Про посилання: Якщо замінити вказівники на звичайні посилання (знак `&` в параметрах функції), то замість оператора стрілки `->` ми зможемо писати звичайну крапку `..`. Код стане безпечнішим, бо посилання на відміну від вказівника не може бути «пустим» (`nullptr`), і програма не вилітає з помилкою сегментації.

Про класи (ООП): Якщо створити клас `TimeTask`, ми сховаємо змінні годин, хвилин і секунд у секцію `private`, а методи роботи з ними зробимо `public`. Тоді ніхто ззовні програми не зможе випадково записати туди неправильні дані (наприклад, 25 годин).

Про юніт-тести: Для автоматичної перевірки крайових випадків у функції AddSec можна використати макрос `assert`. Ми пишемо тести для трьох ситуацій: перехід через північ (23:59:55 + 10 секунд), перехід назад через від'ємний час, та перевірка реакції на початково неправильний час (наприклад, 25 годин).

ЧАСТИНА 3:

Фінальний підрахунок:

Загальна сума балів: $5 + 5 + 5 + 5 + 5 + 5 + 5 + 5 + 4 + 5 + 5 + 5 + 4 = 62$ бали.

Кількість оцінених питань: 13.

Загальна середня оцінка: $62 / 13 = 4.76$ (після округлення — **4.7 з 5**).

Висновок викладача: Робота захищена на майже відмінно. Студент продемонстрував високий рівень розуміння модульного програмування, базових структур даних та керування пам'яттю в C++.